Software Quality Management (CS-7314)

I. McCall's factor

Product operation factors	Product revision factors	Product transition factors
•Correctness	•Maintainability	•Portability
•Reliability	•Flexibility	•Reusability
•Efficiency	•Testability	•Interoperability
•Integrity		
•Usability		

Example

Identify the quality factors for the following requirement document.

(a) The failures frequency of a heart-monitoring unit that will operate in a hospital's intensive care ward is required to be less than one in 20 years. Its heart attack detection function is required to have a failure rate of less than one per million cases. (Reliability)

(b) The software package developed for the Linux operating system should be compatible for applications in a Windows NT environment. (Portability)

(c) GIS SW allowed citizen's access to its GIS through Internet only for viewing and copying data but not to insert changes. (Integrity)

(d) The programming will adhere to the company coding standards and guidelines. (Maintainability)

(e) A staff member should be able to handle at least 60 service calls a day. (Usability/ Efficiency)

II. 5 Marks

1. Software Development Process

   Generic activities in all software processes

   - Specification : What the system should do and its development constraints
   - Development : Production of the software system
   - Validation : Checking that the software is what the customer wants
   - Evolution : Changing the software in response to changing demands

2. Quality Management Activities (or) Quality Assurance, Quality Planning and Quality Control.

   Quality assurance : Establishing organizational quality standards and procedures

   Quality planning : Selecting and modifying applicable quality standards and procedures for a particular project

   Quality control : Ensuring quality standards and procedures are followed by development team

   Quality management should be separate from project management to ensure independence

3. The benefits of Software Quality Assurance.

- Quality assurance helps a company meet its clients' demands and expectations.
- It saves costs and fixes issues.
- It helps to set and maintain quality standard.
- Investing in quality assurance is indispensable in many industries today.
- It is most effective when it's in place from the start.
- It provides confidence, tests the product.
- Benefits of quality assurance are Saves your money, Promotes Organization, Productivity, Efficiency, Prevents Catastrophic Corporate emergencies, Inspires Client Confidence
- Boosts Customer Satisfaction, Brings In More Profit, Great User Experience and Maintains

4. Differentiate between Process Standard and Product Standards

Product Standards	**Process Standards**
☐ Design review form	☐ Design review guidelines
☐ Document naming standards	☐ Document submission procedures
☐ Function prototype format	☐ Version release process
☐ Programming style standards	☐ Project plan approval procedure
☐ Project plan format	☐ Change control process
☐ Change request form	☐ Test data recording procedure

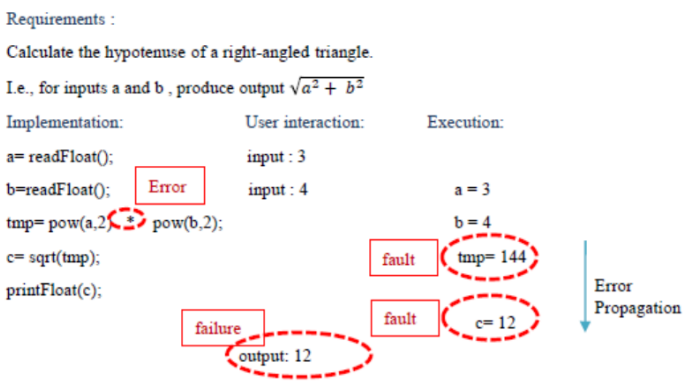
5. The characteristics of the SQA environment process.

The characteristics of the SQA environment process are

(i) Being contracted: Professional software development is almost always contraceted.
(ii) Subjection to customer-supplier relationship: In Professional software development, there is a constant oversight between customers and developer
(iii) Requirement for teamwork: Needteams, Need for a variety of specializations, Need'independent' reviews to ensure quality
(iv) Need for cooperation and coordination with other development teams: Expertise may exist in another team, Need to cooperate with other teams
(v) Need for interfaces with other software systems : Outputs from one system are inputs to another and vice versal.
(vi) Need to continue carrying out a project while the team changes
(vii) Need to continue maintaining the software system for years

6. Software Error, Fault and Failure with example

Error : A mistake, that is made by a human in performing some software activity.
Fault : A fault occurs when a human make an error. A fault is inside view of the system, as seen by the developers.
Failure : A failure is a departure from the system's required behavior. A failure is outside view, a problem that the user sees.



7. Nine causes of software errors.

    1.Faulty requirements definition

    2.Client–developer communication failures

    3.Deliberate deviations from software requirements

    4.Logical design errors

    5.Coding errors

    6. Non-compliance documentation and coding instructions

    7. Shortcomings of the testing process

    8. Procedure errors

    9. Documentation errors

8. McCall's product revision factors.

Maintainability

- Maintainability requirements determine the efforts that will be needed by users.
- Maintenance personnel to identify the reasons for software failures, to correct the failures, and to verify the success of the corrections.
- Refer to the modular structure of software, the internal program documentation.

Flexibility

- The capabilities and efforts required to support adaptive maintenance activities are covered by the flexibility requirements.
- These include the resources required to adapt a software package to a variety of customers of the same trade.
- Support effective maintenance activities.

Testability

- Testability requirements deal with the testing of an information system and its operation.
- Testability requirements for the ease of testing are related to special features in the programs.

9. McCall's product transition factors.

Portability : Portability requirements tend to the adaptation of a software system to other environments, different hardware, different operating systems.

Reusability : Reusability requirements **deal with the use of software modules** designed for a new software project. The reuse of software is expected to

- save development resources
- shorten the development period
- provide higher quality modules

Interoperability : Interoperability requirements focus on creating interfaces with other software systems. Interoperability requirements can specify the name of the software and the output structure accepted as standard in a specific industry or applications area.

10. Staged representation for CMMI model.

- Use maturity levels to measure process improvement.
- Provide sequence of improvements.
- Pre-defined sets of process areas define an improvement path for the organization.
- Maturity levels range from 1 to 5, apply to an organization's overall maturity.
- Provide an easy migration from the SW-CMM to CMMI.
- Provide a single rating that summarizes appraisal results and allow comparisons among organizations.

11. Continuous representation for CMMI model.
    - Use capability levels to measure process improvement.
    - Provide maximum flexibility for focusing on specific process areas.
    - Allow to select the order of improvement.
    - Enable comparisons across and among organizations.
    - Provides an easy migration from EIA 731 to CMMI.
    - Measure maturity of a particular process.
    - Range from 0 through 5.
12. The comparison of staged and continuous representation for CMMI.

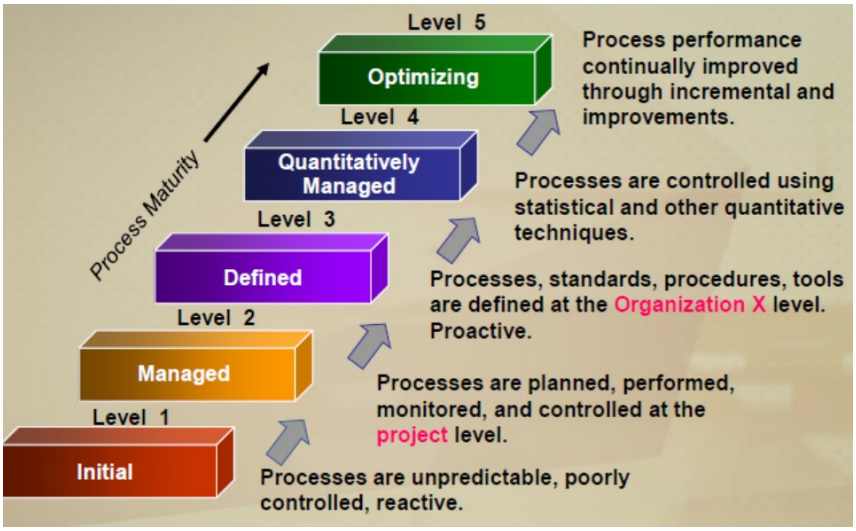
Feature	**Staged Representation**	**Continuous Representation**
Measurement Metric	Process improvement is measured using maturity levels.	Process improvement is measured using capability levels.
Scope of Improvement	Maturity level is the degree of process improvement across a set of process areas.	Capability level is the achievement of process improvement within individual process area.
Focus	Organizational maturity pertains to the "maturity" of processes.	Process area capability pertains to the “maturity” of a particular process.

13. The advantages of Staged representation and Continuous representation

Advantages of Stage representation	Advantages of Continuous representation
• Provides a roadmap for implementing: ▪ groups of process areas ▪ sequencing of implementation • Familiar structure for those transitioning from the Software CMM.	• Provides **maximum flexibility** for focusing on specific process areas • Familiar structure for those transitioning from EIA 731

14. The advantages of CMMI model.
    - Develops efficient process in organizations
    - Allows process improvement in organizations
    - Increase the ability to meet project goals and improve profitability
    - Increased Productivity
    - On Time Deliveries
    - Increased client satisfaction
    - Improved cycle time and quality
    - Increased return on investment
15. Capability Levels and Capability Levels Hierarchy
    - A capability level is a **well-defined evolutionary plateau**.
    - Consists of related specific and generic practices for a process area.
    - Each level is a layer in the foundation for continuous process improvement.
    - Capability levels are **cumulative**.
    - There are **six capability levels** designated by the numbers 0 through 5.
      5. Optimizing/4. Quantitatively Managed/3. Defined/2. Managed/1. Performed/0. Incomplete

16. Five maturity levels for CMMI stages representation.



17. Pros and Cons of Manual Testing

Pros of Manual Testing

- Get fast and accurate visual feedback.
- It is less expensive.
- Human judgment and intuition always benefit the manual element.

Cons of Manual Testing

- It is time-consuming and laborious.
- Less reliable testing method
- Low accuracy result, it is always prone to mistakes and errors.
- It is not possible to reuse the manual test.

18. Pros and Cons of Automated Testing

Pros of automated testing

- A speedy and efficient process
- To reuse and execute the same kind of testing operations
- It works without tiring and fatigue unlike human in manual testing
- It can easily increase productivity
- Automated testing support various applications
- Testing coverage can be increased

Cons of Automated Testing

- It's difficult to get insight into visual aspects of UI like colors, font, sizes, contrast.
- The tools to run automation testing can be expensive.
- Automation testing tool is not yet full proof.
- Debugging the test script is another major issue in the automated testing.
- Test maintenance is costly.

19. Difference between Alpha testing and Beta testing.

Alpha Test	**Beta Test**
• Performed by developers	• Performed by Customers
• It is conducted for software application	• It is conducted for software product
• Performed in Virtual Environment	• Performed in Real Environment
• Involve both black and white box testing	• Involve black testing only

20. Advantage and disadvantage of white box testing.

Advantages

- Enforces the determination of software correctness
- Allows performance of line coverage
- Allows the tester to identify the code that has not yet been executed
- Ensures quality of coding work and apply to coding standards

Disadvantages

- As knowledge of code and internal structure is prerequisite, skilled tester need to carry out type of testing, which increases the cost.
- It is nearly impossible to look into every bit of code to find out hidden errors.

21. The comparison of white box testing and black box testing.

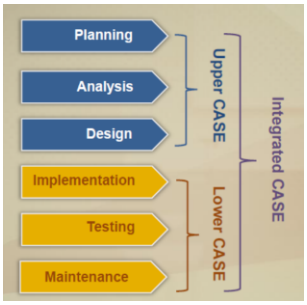
Black Box Testing	**White Box Testing**
Testing method without having knowledge about the actual code or internal structure of the application	Testing method having knowledge about the actual code and internal structure of application
Higher level testing such as functional testing	Testing is performed at a lower level of testing
Concentrates on the functionality of the system under test	Concentrates on the actual code
Black box testing requires requirement specification to test	White box testing requires design documents with data flow diagrams, flowcharts
Black box testing is done by testers	White box testing is done by developers or testers

22. Elements of the Development Plan

- Project products, specifying "Deliverables"
- Project interfaces
- Project methodology and development tools
- Software development standards and procedures
- Map of the development process
- Project milestones
- Project staff organization
- Required development facilities
- Development risks and risk management actions
- Control methods
- Project cost estimates

23. Component of CASE Tools

- Upper CASE tools are used in planning, analysis and design stages of SDLC.
- Lower CASE tools are used in implementation, testing and maintenance.
- Integrated CASE tools are helpful in all the stages of SDLC, from Requirement gathering to Testing and documentation.



24. Differences between software design review, software inspection and walkthrough.

Properties	Design Review	Inspection	Walkthrough
Overview Meeting	No	Yes	No
Participant's Preparations	Yes - thorough	Yes - thorough	Yes - brief
Review session	Yes	Yes	Yes
Follow-up of corrections	Yes	Yes	No
Formal training of participants	No	Yes	No
Participant's use of checklists	No	Yes	No
Error-related Data collection	Not formally required	Formally required	Not formally required
Review documentation	1) Formal design Review report	1) Inspection session findings report 2) Inspection session summary report	1) Walkthrough session findings report

III. 2 Marks

1. Uniqueness of software development process.
   - High complexity, as compared to other industrial products
   - Invisibility of the product
   - Opportunities to detect defects ("bugs") are limited to the product development phase
2. Software quality.

   (IEEE) The degree to which a system,component, or process meets specified requirements.

   The degree to which a system, component, or process meets customer or user needs or expectations.

   (Pressman) Conformance to explicitly stated functional and performance requirements

   Explicitly documented development standards

   Implicit characteristics that are expecte do fall professionally developed software.
3. Software Quality Assurance.
   - Activities that ensure the implementation of processes, procedures, and standards.
   - Focuses on processes and procedures rather than conducting actual testing.
   - Process-oriented activities.
   - Known as Preventive activities.
   - A subset of Software Test Life Cycle (STLC).
4. Quality Control (QC)
   - Activities that ensure the verification of developed software.
   - Focuses on actual testing by executing the software.
   - Product-oriented activities.
   - Corrective activities.
   - QC can be considered as the subset of Quality Assurance.
5. Software Testing
   - The process of evaluating a system
   - To find whether it satisfies the specified requirements or not
   - Executing a system to identify any gaps, errors to the actual desire

6. CMM
   - Stands for Capability Maturity Model
   - A method to evaluate and measure the maturity of the software development process
   - Developed by Carnegie Mellon Uni's Software Engineering Institute (SEI)
   - Similar to ISO 9001, one of the ISO 9000 series of standards
7. Disadvantages of CMMI model.
   - May require additional resources and knowledge to initiate CMMI based process improvement.
   - May require considerable amount of time and effort for implementation.
8. Requirements documents

   A project will be carried out to according to two requirements documents
   - Client's requirements document
   - Developer's additional requirements document
9. Stubs and drivers.
   - A stub replaces an unavailable lower level module, subordinate to the module tested. Stubs are required for top-down testing of incomplete systems. The stub provides the results of calculations the subordinate module. A driver is a substitute module.
   - The driver is passing the test data on to the tested module.Accepting the results calculated by it. Drivers are required in bottom-up testing until the upper level modules are developed.
10. Data Driven Testing

    For testing application functions where the same functions need to be validated with lots of different inputs and large data sets.
11. Regression Testing

    Automated testing is suitable because of frequent code changes and the ability to run the regressions in a timely manner.
12. Incremental Testing
    - Test incrementally.
    - Test the software in modules (unit tests).
    - Test groups of units integrated with new components (integration tests).
    - Repeat until the complete software is done and test again (system tests).
13. Top-down Testing
    - Testing takes place from top to bottom, following the control flow.
    - Components or systems are substituted by stubs.
    - The first module tested is the main module.
    - The lowest level modules are tested last.
14. Bottom-up Testing
    - Testing takes place from the bottom of the control flow upwards.
    - Components or systems are substituted by drivers.
    - The lowest level modules are tested first.
    - Main module is tested last.

15. Usability testing.
    - An area need to measure how user-friendly, efficient, or convenient the software is for the end users.
    - Human observation is the most important factor.
    - Manual approach is preferable.
16. Root causes of software errors

    Usually considered the **root cause** of software errors
    - **Incorrect requirement :** Simply stated, 'wrong' definitions
    - **Incomplete :** Unclear or implied requirements
    - **Missing requirements :** Just flat-out 'missing'
    - **Inclusion of unneeded :** Impacts budgets, complexity, development time
17. Documentation errors
    - Errors in the design documents
    - Errors in the documentation in the User Manuals, Operators Manual, other manuals
    - Errors in on-line help, if available
    - Listing of non-existing software functions
    - Many error messages are totally meaningless
18. Software development standards and procedures

    Software development STANDARD
    - ISO
    - CMMI

    Software Development PROCEDURE
    - Processes
    - Tasks
    - WBS: Work Breakdown Structure
19. Element of quality plan.

    List of quality goals

    Review activities

    Software tests

    Acceptance tests for software externally developed

    Configuration management tools and procedures
20. Review Activities

    The quality plan should provide a complete listing of all planned review activities:
    - designreviews(DRs),
    - designinspections,
    - codeinspections
21. The criteria are needed in development and quality plans for small projects and internal projects.

    The Development plan
    - Estimates of project costs
    - Development risks
    - Project benchmarks
    - Project products, indicating "deliverables"

    The quality plan
    - Quality goals
22. Software Development Risks
    - Scheduling and timing risks
    - System functionality risks
    - Subcontracting risks
    - Requirement management risks
    - Resource usage and performance risks
    - Personnel management risks

23. CASE

    CASE stands for Computer Aided Software Engineering.
    It means development and maintenance of software projects with help of various automated software tools. CASE tools stands for computerized software development tool.

24. CASE Tools

    CASE tools stands for computerized software development tool.
    - support the developer when performing one or more phases of the software life cycle
    - support software maintenance
    - Classic CASE tools
        - Interactive debuggers, compilers, project progress control systems
    - Real CASE tools
        - Support several phases of development

25. The review process.

    "a process or meeting during which a work product or set of work products is presented to project personnel, managers, users, customers, or other interested parties for comment or approval."
    IEEE : Essential to detect/correct errors in these earlier work products because the cost of errors downstream is very expensive!

26. Indirect objectives of the review.

    Indirect objectives - are more general in nature
    - To provide an informal meeting place for exchange of professional knowledge about methods, tools and techniques.
    - To record analysis and design errors that will serve as a basis for future corrective actions.

27. Direct objectives of the review.

    Direct objectives - Deal with the current project
    - To detect analysis and design errors
    - To identify new risks
    - To locate deviations from templates
    - To approve the analysis or design product
      Approval allows the team to continue on to the next development phase

28. Inspection

    - Emphasize the objective of corrective action, more formal
    - Look to improve methods as well
    - Consider to contribute more to general level of SQA

29. Post-Review Activities

    - Prepare the review report, including the action items.
    - Establish follow-up to ensure the satisfactory performance of all the corrections included in the list of action items.